



Universidad de Oviedo
Grado en Física

Introducción a la Física Computacional

Curso 2023–24

Python 3 como lenguaje algorítmico: Tercera parte

Índice general

6. Python 3 como lenguaje algorítmico: Tercera parte	2
6.1. Objetivo	2
6.2. Qué es una función en Python	2
6.3. Funciones de conversión de tipos	3
6.4. Funciones matemáticas	4
6.5. Funciones definidas por el usuario	4
6.5.1. Argumentos de funciones	6
6.5.2. Paso de argumentos por valor y por referencia	10
6.5.3. Funciones que devuelven valores	12
6.5.4. Funciones como argumentos	13
6.5.5. Funciones lambda	14
6.6. Variables globales y locales	15
6.7. Módulos	17
6.8. Algunos módulos estándar	20
6.8.1. Módulo <code>math</code>	21
6.8.2. Módulo <code>time</code>	23
6.8.3. Módulo <code>sys</code>	24
6.8.4. Módulo <code>os</code>	25

Práctica 6

Python 3 como lenguaje algorítmico: Tercera parte

6.1. Objetivo

Familiarizarse con el uso de funciones y módulos en Python.

6.2. Qué es una función en Python

Una función es un fragmento de código con un nombre asociado que realiza una serie de tareas y devuelve un valor. A los fragmentos de código que tienen un nombre asociado y no devuelven valores se les suele llamar “procedimientos”. En Python no existen los procedimientos, ya que cuando el programador no especifica un valor de retorno la función devuelve el valor `None` (nada).

El uso de funciones permite racionalizar el código: como función, programada de manera general, una tarea repetitiva dependiente de algunas variables (denominados “argumentos” de la función), se puede usar para cualquier problema del mismo tipo.

Por ejemplo, podemos diseñar una función que resuelva una ecuación de segundo grado a partir de sus parámetros a , b y c , devolviéndonos como resultado las raíces (reales o complejas). Esta función general nos permite resolver **cualquier** ecuación de segundo grado.

Como veremos más adelante, Python permite agrupar bajo un mismo “paraguas” diversas funciones relacionadas entre sí, en lo que se denominan **módulos**.

6.3. Funciones de conversión de tipos

Hemos visto que los números de diferentes tipos se convierten automáticamente a un tipo común después de la operación aritmética o una comparación. También se pueden cambiar los tipos con las funciones:

- `int(a)`: convierte `a` en un entero.
- `float(a)`: convierte `a` en un `float`.
- `complex(a)`: convierte `a` en un complejo `a+0j`.
- `complex(a,b)`: convierte `a` en un complejo `a+bj`.

Las funciones anteriores sólo trabajan convirtiendo cadenas en números tan largos como las cadenas de manera que representen un número válido. La conversión de `float` a entero se realiza con un proceso de truncamiento, no por redondeo.

Ejemplo 6.3.1. *Uso de las funciones de conversión de tipos.*

En el modo interactivo de Python escriba:

```
>>> a = 5

>>> b = -3.6

>>> d = '4'

>>> print(int(b))

-3

>>> print(float(d))

4.0

>>> print(complex(a,b))

(5-3.6j)

>>> print(int(d))

4

>>> print(float(d))

4.0
```

6.4. Funciones matemáticas

El núcleo de Python sólo contiene las siguientes funciones matemáticas:

- `abs(a)`: valor absoluto de `a`.
- `max(secuencia)`: el mayor elemento de una secuencia (tupla, lista o cadena de caracteres).
- `min(secuencia)`: el menor elemento de una secuencia.
- `round(a, n)`: redondea `a` con `n` decimales.

La mayoría de las funciones matemáticas se encuentran en el módulo `math` que estudiaremos más adelante.

6.5. Funciones definidas por el usuario

Es posible crear nuevas funciones para resolver problemas concretos. En Python la definición (o declaración) de una función se realiza de la manera siguiente:

```
def nombre_funcion (param1, param2, ...):  
    sentencia 1  
    sentencia 2  
    ...  
    return valor_devuelto
```

donde `param1`, `param2`, ..., son los **parámetros** (también denominados **argumentos formales**). Obsérvese también el **indentado** uniforme.

`valor_devuelto`, aparte de un valor numérico, cadena, etc., también puede ser una tupla, lista, etc. Cuando sea una tupla no hace falta poner los paréntesis.

También podemos encontrarnos con una cadena de texto como primera línea del cuerpo de la función. Estas cadenas se conocen con el nombre de *docstring* ("cadena de documentación") y, como su nombre indica, sirven de documentación de la función:

```
def iden (p1, p2):  
    '''  
    Esta función imprime los dos valores pasados como argumentos  
    '''  
    print(p1)  
    print(p2)
```

Al **definir** una función lo único que hacemos es asociar un nombre al fragmento de código que conforma la función, de forma que podamos ejecutar dicho código más tarde referenciándolo por su

nombre. Es decir, a la hora de escribir estas líneas no se ejecuta la función. Para **llamar** (o invocar) a la función (ejecutar su código) se escribiría el nombre de la función a la que queremos llamar seguido de los valores que queramos pasar como argumentos entre paréntesis.

En Python un parámetro puede ser cualquier objeto, incluida una función. Se pueden definir funciones que no tengan parámetros y/o que tampoco devuelvan valores.

Ejemplo 6.5.1. *Uso de funciones definidas por el usuario que no tienen argumentos y que no devuelven ningún valor.*

1. Escriba en un fichero denominado `lineapuntos.py` el siguiente código:

```
'''
Ejemplo de uso de funciones sin argumentos y sin valor de retorno
'''

def linea():
    '''
    Imprime un montón de guiones
    '''
    print('-----')

def puntos():
    '''
    Imprime un montón de puntos
    '''
    print('.'*25)

print('Invocamos primero a la función línea y luego a puntos')
linea()
puntos()

print('Ahora vamos a llamar a la función línea dos veces')
linea()
linea()

print('Ahora vamos a llamar a la función puntos tres veces')
puntos()
puntos()
puntos()
```

2. Guarde el fichero, ejecute el programa y observe la respuesta.

6.5.1. Argumentos de funciones

Se pueden construir funciones que tomen uno o más argumentos pero que no devuelvan ningún valor.

Los argumentos que se pasan cuando se invoca la función se denominan **argumentos actuales** (o **argumentos** a secas). El número de valores que se pasan como argumentos actuales al llamar a la función tiene que coincidir con el número de argumentos formales que la función acepta según la declaración de la función. En caso contrario Python dará un error.

Ejemplo 6.5.2. *Uso de funciones definidas por el usuario con argumentos que no devuelven ningún valor.*

1. Escriba en un fichero denominado `sin_retorno.py` el siguiente código:

```
'''
Uso de funciones definidas por el usuario con varios argumentos,
pero que no devuelven ningun valor.
'''

def imprime(p1, p2):
    '''
    Imprime los dos argumentos, uno de debajo del otro
    '''
    print(p1)
    print(p2)

# Aquí empieza el cuerpo principal del programa

# Definimos dos variables de texto
nombre = 'Albert'
apellido = 'Einstein'

# Llamamos a la funcion iden pasandole esas variables como argumentos
imprime(nombre, apellido)
print('---\n')

# Llamamos a la funcion pasandole constantes como argumentos
imprime('Isaac', 'Newton')
print('---\n')

# Incluso podemos mezclar
imprime(nombre, 'Peral')
print('---\n')

# Otro ejemplo con variables con el mismo nombre que los parámetros
```

```
# de la función. Ojo con confundirlos.
p1='Richard'
p2='Maxwell'
imprime(p1,p2)
print('---\n')

# Última invocación con un solo parámetro. ¿Qué pasa en este caso?
imprime('Richard')
```

2. Guarde el fichero, ejecute el programa y observe la respuesta.

También se pueden usar argumentos con **valores por defecto**, que se indican en la definición de la función.

Los argumentos de invocación se asocian a los parámetros de la definición por **posición**.

Ejemplo 6.5.3. Asociación de parámetros por posición:

1. Escriba en el fichero `posicion.py` el siguiente código:

```
'''
Ejemplo de función invocada con varios parámetros por posición
'''

def posicion(x0, v0, a, t):
    '''
    Imprime la posición de un móvil uniformemente acelerado
    Argumentos:
    * x0: Posición inicial
    * v0: Velocidad inicial
    * a: Aceleración
    * t: Instante de tiempo
    '''
    print(x0 + v0*t + 0.5*a*t**2)

# Usamos la función
posicion(2, 3, 4, 5)
```

La función interpreta que $x_0 = 2$, $v_0 = 3$, $a = 4$, $t = 5$.

2. Ejecute el programa y observe la respuesta.
3. Modifique la función para que imprima el valor de la posición con formato de coma flotante con dos decimales, y el valor de cada uno de los parámetros.

Los **valores por defecto** para los argumentos se definen situando un signo igual después del nombre del argumento y a continuación el valor por defecto.

Ejemplo 6.5.4. Argumentos con valores por defecto

1. Escriba en el fichero `arg_defecto.py` el siguiente código:

```
'''
Programa con una función con argumentos con valores por defecto
'''

def imprimir(texto, veces=1):
    '''
    Esta función imprime el texto repetido varias <veces>
    '''
    print(voces*texto)

# La usamos normalmente (repite tres veces el texto)
imprimir("Hola 3 veces ", 3)

# Aprovechamos los argumentos por defecto
# (usa el valor por defecto del argumento <veces>)
imprimir("Hola 1 vez por defecto")

# Lo anterior es equivalente a...
imprimir("Hola 1 vez explicito", 1)
```

2. Ejecute el programa y observe la respuesta.
3. Al invocar la función `imprimir`, si solo se especifica el primer parámetro, se asigna el valor `1` al argumento `veces`.

Es posible definir funciones con un número variable de argumentos. Para ello se usa un parámetro final cuyo nombre debe ir precedido del carácter `*`:

Ejemplo 6.5.5. Definición de una función con un número variable de argumentos.

1. Escriba en el fichero `arg_variable.py` el siguiente código:

```
'''
Programa con una función con un número variable de argumentos
'''

def varios(par1,par2,*otros):
    '''
    Esta función crea una tupla de nombre otros en la que se
    almacenan los valores de todos los parámetros extra pasados
    como argumentos.
    Imprime la tupla para comprobarlo e imprime cada uno de los
    valores guardados en la tupla utilizando un bucle for.
    '''
    print(otros) # imprime los valores extra guardados en la tupla
    for valores in otros: # bucle para recorrer la tupla
        print(valores)

# La invocamos sin parámetros extra, en este caso
# imprime la tupla otros vacía
# y en el bucle no imprime nada.
print('No hay parámetros extra')
varios(1,2)

# La invocamos con parámetros extra.
print('Los parámetros extra son:')
varios(1,2,3,4,5,6)

# La invocamos con parámetros extra diferentes.
print('Los parámetros extra son:')
varios(1,2,3,(1,2,3),'hola',[4,5,6])
```

2. Ejecute el programa y observe la respuesta.
3. Con este método la función crea una tupla (de nombre `otros` en el ejemplo) en la que se almacenan los valores de todos los parámetros extra pasados como argumentos.

También se puede preceder el nombre del último argumento con `**`, en cuyo caso en lugar de una tupla se utilizaría un diccionario. Las claves de este diccionario serían los nombres de los argumentos indicados al llamar a la función y los valores del diccionario, los valores asociados a estos argumentos.

Ejemplo 6.5.6. En el siguiente ejemplo se utiliza la función `items` de los diccionarios, que devuelve una lista con sus elementos, para imprimir los argumentos que contiene el diccionario.

1. Escriba en el fichero `arg_variable_dicc.py` el siguiente código:

```
'''
Programa con una función con un número variable de argumentos
'''

def varios(par1,par2,**mas):
    '''
    Esta función crea un diccionario de nombre mas en la que se
    almacenan los valores de todos los parámetros extra pasados
    como argumentos.
    Imprime el diccionario para comprobarlo e imprime cada uno
    de los valores guardados en el diccionario utilizando un
    bucle for y el método items de diccionarios.
    '''
    print(mas) # imprime los valores extra guardados en el
               # diccionario
    # mas.items() devuelve una lista de tuplas,
    # el primer elemento de cada tupla es la clave y el
    # segundo elemento es el valor de las variables guardadas
    # en el diccionario.
    for clave, valor in mas.items(): #recorre el diccionario
        print('la clave es %s y el valor es %d' %(clave,valor))

# La invocamos sin parámetros extra, en este caso
# imprime el diccionario está vacío
# y en el bucle no imprime nada.
print('No hay parámetros extra')
varios(1,2)

# La invocamos con parámetros extra.
print('Los parámetros extra son:')
varios(1,2,tres=3,cuatro=4,ocho=8,diez=10)
```

2. Ejecute el programa y observe la respuesta.

6.5.2. Paso de argumentos por valor y por referencia

No todos los cambios que hagamos a los parámetros dentro de una función Python se reflejarán fuera de esta. En Python existen objetos inmutables, como las tuplas, las cadenas y los números, por lo que si intentáramos modificar una tupla pasada como argumento lo que ocurriría en realidad es que se crearía una nueva instancia, por lo que los cambios no se verían fuera de la función (aunque sí dentro de ella).

Dependiendo del tipo de dato que le pasemos a la función cuando la invocamos podemos diferenciar dos comportamientos:

- Paso por valor: Se crea una copia local del valor que pasamos dentro de la función. La función actúa sobre otra dirección de memoria que desaparece cuando termina la función. Las modificaciones que hagamos, por tanto, no se reflejarán fuera de la función. Los tipos simples se pasan por valor: `int`, `float`, `complex`, `bool`,...
- Paso por referencia: A la función le llega la (dirección de memoria de la) propia variable que se pasa. Por tanto, los cambios que se realicen sobre ese argumento dentro de la función se verán también fuera. Las colecciones y los tipos compuestos se pasan por referencia: Tuplas, listas, diccionarios, ...

Ejemplo 6.5.7. En este ejemplo se hace uso de la función `append` de las listas.

1. Escriba el siguiente código en el fichero `valor_y_referencia.py`

```
'''
Ejemplo para demostrar cuándo un tipo pasa por valor y cuándo pasa
por referencia. Al ejecutarlo imprimirá:

---
Valores de x e y al principio: 10 [200]
Valores de x e y al empezar f: 10 [200]
Valores de x e y al terminar f: 15 [200, 220]
Valores de x e y al final (fuera de f): 10 [200, 220]
---
'''

def f(x, y):
    '''
    Esta función incrementa en 3 el primer parámetro y asume
    que el segundo es una lista a la que le añade otro valor.
    '''
    print('Valores de x e y al empezar f:', x, y)
    x = x + 3
    y.append(220)
    print('Valores de x e y al terminar f:', x, y)

x = 10
y = [200]
print('Valores de x e y al principio:', x, y)
f(x,y)
print('Valores de x e y al final (fuera de f):', x, y)
```

2. Ejecute el programa y observe la respuesta.

La variable `x` no conserva los cambios una vez se sale de la función porque los enteros son inmutables en Python. Sin embargo la variable `y` si los conserva, porque las listas son mutables.

6.5.3. Funciones que devuelven valores

Para que una función devuelva un valor debemos usar la sentencia `return`.

Ejemplo 6.5.8. Esta función calcula la altura máxima alcanzada en un lanzamiento vertical hacia arriba y devuelve el resultado como valor de retorno.

1. Escriba en un fichero (`y_max.py`) el siguiente código:

```
'''
Ejemplo del uso de return en una función.
'''
def y_max(v0):
    '''
    Esta funcion calcula la altura maxima alcanzada en un
    lanzamiento vertical hacia arriba
    '''
    return v0**2/(2*9.81)

# Preguntamos la velocidad inicial
velocidad = eval(input('velocidad inicial? '))
# Calculamos la altura máxima
hmax = y_max(velocidad)
# Imprimimos el resultado
print('Para una velocidad inicial de %5.2f m/s' % velocidad)
print('la altura maxima alcanzada es: %5.2f m ' % hmax)
```

2. Ejecute el programa y observe la respuesta.

Se pueden devolver varios valores con `return` separándolos por comas. Python crea una tupla cuyos elementos son los valores a devolver y esta tupla es la que se devuelve.

Ejemplo 6.5.9. Función que devuelve varios valores.

1. Escriba en un fichero (`cuadrados.py`) el siguiente código:

```

'''
Ejemplo de programa con una función que devuelve más de un valor
'''
def cuadrados(x,y):
    '''
    Devuelve los cuadrados de los parámetros
    '''
    return x**2, y**2

# Pedimos dos valores
x = eval(input('Introduce x: '))
y = eval(input('Introduce y: '))

# Una forma de usar la función
print('Calculamos los cuadrados....')
x2, y2 = cuadrados(x,y)
print('%f al cuadrado es %f'%(x,x2))
print('%f al cuadrado es %f'%(y,y2))

# Otra forma de obtener la información de la función (como tupla)
print('Calculamos los cuadrados otra vez....')
respuesta = cuadrados(x,y)
print('La respuesta es de tipo ' + str(type(respuesta)))
print('La respuesta es ' + str(respuesta))
print('%f al cuadrado es %f'%(x,respuesta[0]))
print('%f al cuadrado es %f'%(y,respuesta[1]))

```

2. Ejecute el programa y observe la respuesta.

Ejercicio 6.5.1. Modifique el ejemplo 6.5.3 creando una función `mrúa(x0,v0,a,t)` que devuelva los valores de posición, velocidad y aceleración para un movimiento rectilíneo uniformemente acelerado.

Ejercicio 6.5.2. Defina una función que calcule las raíces de la ecuación de segundo grado $ax^2 + bx + c = 0$ en función del signo de su discriminante $b^2 - 4ac$. Escriba un programa que pida los tres parámetros a , b y c de la ecuación y que llame a la función para calcular las raíces. Compruebe que las respuestas son correctas.

6.5.4. Funciones como argumentos

El argumento de una función también puede ser otra función.

Ejemplo 6.5.10. *Uso de funciones como parámetros. En este ejemplo utilizaremos una función que se pasará como parámetro de otra.*

1. Escriba un fichero `derivadas.py` con el siguiente código:

```
'''
Ejemplo de funciones usadas como argumentos de otras funciones
'''

def cuadrado(x):
    return x**2

def finite_diff(f, x, h = 0.0001):
    '''
    Aproxima la primera y segunda derivada de f en x
    basándose en la fórmula:
                f(x0+h) - f(x0-h)
    f'(x0) = lim -----
                h->0      2h
    '''
    fm, f0, fp = f(x-h), f(x), f(x+h)
    df, ddf = (fp-fm)/(2*h), (fp-2*f0+fm)/h**2
    return df, ddf

x0 = 0.5
df, ddf = finite_diff(cuadrado, x0)
print('La 1ª derivada del x² en x0=' + str(x0) + ' vale: ' + str(df))
print('La 2ª derivada del x² en x0=' + str(x0) + ' vale: ' + str(ddf))
```

2. Guarde el fichero, ejecute el programa y compruebe la respuesta.

En el ejemplo anterior la función `cuadrado` se pasa a `finite_diff` como argumento.

6.5.5. Funciones lambda

Si la función tiene la forma de una expresión, puede ser definida de forma sencilla con la sentencia `lambda`, que permite funciones de una sola línea:

```
nombre_funcion = lambda arg1, arg2, ...: expresion
```

Ejemplo 6.5.11. *Uso de la sentencia `lambda`.*

1. Escriba un fichero `funcion_lambda.py` con el siguiente código:

```
'''
Programa que define una función de dos formas diferentes
'''

def suma_cuadrados1(x,y):
    '''
    Esta función calcula los cuadrados de los valores
    pasados como argumentos y devuelve su suma.
    '''
    return x**2+y**2

# Definimos la función anterior con la sentencia lambda

suma_cuadrados2=lambda x,y:x**2+y**2

# Solicitamos datos

x=eval(input('x?'))
y=eval(input('y?'))

# Invocamos la primera función

resultado1=suma_cuadrados1(x,y)

print('El resultado de la primera función es: '+ str(resultado1))

# Invocamos la segunda función

resultado2=suma_cuadrados2(x,y)

print('El resultado de la segunda función es: '+ str(resultado2))
```

2. Guarde el fichero, ejecute el programa y compruebe la respuesta.

6.6. Variables globales y locales

Las variables definidas en un fichero fuera de una función son **globales** y son conocidas a partir del punto en el que son definidas dentro del fichero (incluso dentro de las funciones).

Sin embargo, si dentro de una función (o un bloque) se realiza una asignación a una variable con el mismo nombre (identificador), se creará una nueva variable **local** que *ocultará* a la global impidiendo el acceso a la misma.

Si se precisa modificar el valor de una variable global en el interior de una función, se debe hacer uso de la sentencia `global` antes de cualquier asignación.

Ejemplo 6.6.1. *Uso de variables globales y locales.*

1. Escriba en el fichero `noglobal.py` el siguiente código:

```
'''
Ejemplo del ámbito de validez de variables
'''

x = 3 # Variable global

def f1(y):
    '''
    Esta función solo depende de < y >.
    Puesto que no usamos ninguna variable local llamada < x >,
    se usa la variable global < x > (si existe)
    La < x > global es accesible (solo para obtener su valor).
    '''
    print('En f1: ', x, y)

def f2(y):
    '''
    Esta función solo depende de < y >.
    Al asignar un valor a una variable de nombre < x >, se crea una
    nueva variable local con ese nombre que solo es conocida y válida
    dentro de f2.
    La < x > global es ocultada por la < x > local.
    '''

    # Define una nueva variable cuyo identificador es < x > válida
    # solamente en esta función
    x = 8

    # Usa la < x > local. No hay acceso a la otra
    print('En f2: ', x, y)

def f3(y):
    '''
    Esta función solo depende de < y >.
    Al declarar una variable < x > como global indicamos que en el
    interior de la función ese identificador se refiere a la variable
    < x > global (a la de fuera) y que por tanto, cualquier modificación
    que hagamos actúa sobre ella.
    '''
```

```
La < x > global es, por tanto, accesible. Y podemos modificar su  
valor.  
'''  
global x  
  
# Usa la < x > global. Podemos modificarla  
x = 8  
  
print('En f2: ', x, y)  
  
# Probamos f1  
print('Antes de llamar a f1, x = ' + str(x))  
f1('a')  
print('Después de llamar a f1, x = ' + str(x))  
  
# Probamos f2  
print('\nAntes de llamar a f2, x = ' + str(x))  
f2('b')  
print('Después de llamar a f2, x = ' + str(x))  
  
# Probamos f3  
print('\nAntes de llamar a f3, x = ' + str(x))  
f3('b')  
print('Después de llamar a f3, x = ' + str(x))
```

2. Ejecute el programa y observe la respuesta.

En el ejemplo anterior, la función `f3` cambia el valor de la variable global. Se dice entonces que existe un *efecto lateral* y debe tenerse especial cuidado. Es muy recomendable tratar de evitar este tipo de situaciones.

Se denomina *función pura* aquélla que no tiene efectos laterales (no usa variables globales, ni paso por referencia, ni maneja ficheros).

6.7. Módulos

Las funciones relacionadas entre sí se pueden agrupar en módulos. Hay muchos (cientos) módulos preparados por desarrolladores para realizar tareas relacionadas entre sí. Un módulo también puede contener **submódulos**. En ese caso el módulo principal se denomina **paquete** ('package').

Excepto en el caso de programas muy simples, es conveniente subdividir las tareas en funciones asociadas en módulos.

Ejemplo 6.7.1. *Pidiendo ayuda sobre módulos.*

Escriba en el modo interactivo de Python:

```
>>> help('modules')
```

Nos dará un listado de los módulos de Python desarrollados. Si quiere salir de la ayuda presione la letra **q**. La ayuda de un módulo concreto se obtiene así:

```
>>> help(nombre_del_módulo)
```

por ejemplo:

```
>>> help('time')
```

nos da ayuda acerca de funciones sobre tiempo y fecha.

Un programador puede diseñar un módulo juntando funciones en un mismo fichero. El nombre del fichero pasará a ser el nombre del módulo.

Ejemplo 6.7.2. *En este ejemplo se crea un fichero `aritmetica.py` que puede usarse como módulo. Define dos variables (cero y uno) y dos funciones (suma y resta).*

1. Escriba el siguiente código en el fichero `aritmetica.py`:

```
'''
    Aquí describimos el módulo.
    Este módulo define algunas funciones y variables
'''

uno = 1
cero = 0

def suma (x, y):
    '''
        Suma valores
    '''
    return x+y

def resta (x, y):
    '''
        Resta valores
    '''
    return x-y
```

2. Guarde el fichero.

3. Para hacer conocido todo lo definido en el fichero `aritmetica.py` en el modo interactivo de Python debe escribir:

```
>>> from aritmetica import *
```

4. Ahora ya se pueden invocar todas las funciones definidas en el módulo `aritmetica`.

```
>>> print(unos)
1
>>> print(cero)
0
>>> suma(3, 4)
7
>>> resta(3, 4)
-1
```

5. Otra forma de hacer conocido todo lo definido en el fichero `aritmetica.py` es:

```
>>> import aritmetica
```

pero de esta forma la invocación de cada función necesita ser precedida del nombre del módulo al que pertenece (en este caso `aritmetica`), por lo que para invocar ahora a las funciones se debe escribir:

```
>>> print(aritmetica.unos)
1
>>> print(aritmetica.cero)
0
>>> aritmetica.suma(3, 4)
7
>>> aritmetica.resta(3, 4)
-1
```

La primera manera de utilizar un módulo, `from aritmetica import *`, hace que las variables y funciones del módulo pasen a ser conocidas por su nombre exacto (por ejemplo `suma` y muchas otras más en general, previamente desconocidas en el programa). Esto puede ser peligroso puesto que cualquier variable con el nombre `suma` (por ejemplo) que existiera previamente antes de `from aritmetica import *` desaparecerá.

El segundo método, `import aritmetica`, es más seguro, pero su uso es más engorroso. Sólo se perdería una variable `aritmetica` si ya existía previamente. Para evitar teclear tanto al usar las funciones dentro del módulo aritmética se puede usar un "alias": `import aritmetica as ari`. De esa manera se invocaría `ari.suma`, etc. (el alias es `ari`).

También se pueden cargar funciones y variables concretas usando:

```
>>> from aritmetica import cero, suma
```

En este caso ni `unos` ni `resta` están disponibles para usarse.

El **contenido** de un módulo (o sea las variables y funciones que 'exporta') puede ser mostrado con la sentencia `dir(modulo)`. Se puede ver el contenido del módulo del ejemplo anterior escribiendo:

```
>>> import aritmetica
>>> dir(aritmetica)
```

Se pide ayuda sobre el contenido de un módulo mediante `help(modulo)`.

```
>>> import aritmetica
>>> help(aritmetica)
```

la salida que nos proporcionará dicha sentencia será de la forma:

```
Help on module aritmetica:

NAME
    aritmetica

DESCRIPTION
    Aquí describimos el módulo.
    Este módulo define algunas funciones y variables

FUNCTIONS
    resta(x, y)
        Resta valores

    suma(x, y)
        Suma valores

DATA
    cero = 0
    uno = 1

FILE
    /tmp/aritmetica.py
```

Ejercicio 6.7.1. Con la función `finite_diff` definida en el ejemplo 6.5.10 cree una nueva función que calcule la velocidad y la aceleración de un móvil con movimiento rectilíneo a partir de la función que define la posición con respecto al tiempo de dicho móvil. Considere para la posición una función polinómica del tiempo. Para realizar el ejercicio cree un módulo llamado `cinematica_rectilineo` que contenga todas las funciones necesarias.

6.8. Algunos módulos estándar

Python viene con una biblioteca de módulos estándar. Algunos módulos son internos al intérprete y proporcionan acceso a las operaciones que no son parte del núcleo del lenguaje pero se han incluido por eficiencia o para proporcionar acceso a primitivas del sistema operativo, como las llamadas al sistema. El conjunto de dichos módulos es una opción de configuración. Algunos de estos módulos son:

- `math`: la mayoría de las funciones matemáticas en Python se encuentran en este módulo.

- `time`: nos da información sobre tiempo y fecha.
- `sys`: nos permite interactuar con el sistema operativo.
- `os`: nos permite interactuar con el sistema operativo con más funciones que el anterior.

6.8.1. Módulo `math`

En Python la mayoría de las funciones matemáticas se encuentran en el módulo `math`. Existen tres formas de acceder a las funciones dentro del módulo:

- Cargado todas las funciones del módulo con la sintaxis:

```
from math import *
```
- Seleccionando las funciones que queremos usar del módulo:

```
from math import func1, func2
```
- Habilitando el módulo (método utilizado por la mayoría de los programadores) de la forma:

```
import math
```

Dentro de `math` tenemos las funciones: `acos`, `acosh`, `asin`, `asinh`, `atan`, `atan2`, `atanh`, `ceil`, `copysign`, `cos`, `cosh`, `degrees`, `erf`, `erfc`, `exp`, `expm1`, `fabs`, `factorial`, `floor`, `fmod`, `frexp`, `fsum`, `gamma`, `hypot`, `isinf`, `isnan`, `ldexp`, `lgamma`, `log`, `log10`, `log1p`, `modf`, `pow`, `radians`, `sin`, `sinh`, `sqrt`, `tan`, `tanh`, `trunc`

y las variables:

```
e = 2.718281828459045
```

```
pi = 3.141592653589793
```

```
tau = 6.283185307179586 #  $\tau = 2 \cdot \pi$ 
```

Ojo con asignar otros valores a estas variables: está permitido pero se pierden sus valores esperados matemáticamente.

Ejemplo 6.8.1. *Uso del módulo `math`. En el siguiente ejemplo se define una función `maxmin(f, a, b, n=1000)` que devuelve los valores máximo y mínimo de la función $f(x)$ (evaluada en n puntos) en el intervalo (a, b) .*

1. Escriba en un fichero (`maxmin.py`) el siguiente código:

```
'''
Ejemplo de función matemática que devuelve los valores máximo y mínimo
de una función en un intervalo
'''

from math import cos, pi

def maxmin(f, a, b, n=1001):
    '''
    Esta función devuelve el máximo y el mínimo de la función <f> en el
    intervalo de extremos <a> y <b> muestreándola en <n> puntos.
    '''
    h = (b-a)/float(n-1) # el paso
    X = [a+i*h for i in range(n)] # comprension de lista
    Y = [f(x) for x in X] # comprension de lista
    return max(Y), min(Y)

maximo, minimo = maxmin(cos, -pi/2, 2*pi)
print('Máximo de coseno entre -90º y 90º: ' + str(maximo))
print('Mínimo de coseno entre -90º y 90º: ' + str(minimo))
```

2. Ejecute el programa y observe la respuesta.

Ejercicio 6.8.1. Modifique el ejemplo 6.5.10 para que calcule la primera derivada de las siguientes funciones:

1. $f(x) = e^x$ en $x = 0$,
2. $f(x) = e^{-2x}$ en $x = 0$,
3. $f(x) = \cos x$ en $x = 2\pi$,
4. $f(x) = \ln x$ en $x = 1$.

Use $h = 0.01$. En cada caso, haga que el programa escriba la diferencia entre el valor exacto de la derivada (conocido analíticamente) y el resultado dado por la derivada numérica.

Ejercicio 6.8.2. La integral de una función $f(x)$ en un intervalo $[a, b]$ puede realizarse en primera aproximación dibujando una línea recta que una los puntos $(a, f(a))$ y $(b, f(b))$, y calculando el área del trapecio bajo la línea de la forma:

$$\int_a^b f(x) \, dx \simeq \frac{b-a}{2} [f(a) + f(b)]$$

Defina una función `integral(f, a, b)` que devuelva el valor aproximado de la integral mediante la ecuación anterior de las siguientes funciones:

1. $\int_0^\pi \cos x \, dx$,
2. $\int_0^\pi \sin x \, dx$,
3. $\int_0^{\pi/2} \sin x \, dx$

En cada caso, haga que el programa escriba la diferencia entre el valor exacto de la integral (conocido analíticamente) y el resultado dado por la integral numérica.

Ejercicio 6.8.3. Se puede mejorar fácilmente la aproximación de la integral utilizada en el ejercicio anterior aproximando la función $f(x)$ mediante una primera línea que una $(a, f(a))$ con el punto medio $(c, f(c))$ entre a y b y otra segunda línea que una el punto medio $(c, f(c))$ con $(b, f(b))$. El punto medio c es igual a $\frac{1}{2}(a + b)$. El área bajo estas dos líneas es el área de dos trapecios que nos proporcionarán una nueva expresión aproximada de la integral de $f(x)$. Obtenga la nueva expresión aproximada de la integral e implemente una nueva función en **Python** que calcule de nuevo las integrales aproximadas de las funciones del ejemplo anterior.

Ejercicio 6.8.4. Defina una nueva función que calcule la integral utilizando n trapecios. La variable n será un argumento de la función que podrá tener un valor asignado por defecto.

6.8.2. Módulo `time`

Este módulo nos proporciona funciones para el manejo de fechas y del tiempo. Algunas interesantes son:

- `time.time()` nos devuelve el número de segundos desde un instante inicial (dependiente del sistema operativo).
- `time.ctime(x)` nos devuelve una cadena con la fecha que corresponde al tiempo `x` en segundos. Si no se indica `x` se usa la fecha y hora actuales.
- `time.sleep(x)` suspende la ejecución durante `x` segundos.
- `time.process_time()` (en versiones antiguas `time.clock()`) nos devuelve el tiempo de ejecución de CPU hasta ese momento del intérprete de Python.

Ejemplo 6.8.2. *Ejemplo de uso de `time.clock()`*

1. Escriba en un fichero con el nombre `pruebas.py`:

```
'''  
Ejemplo de uso de time.clock() para medir cuanto tiempo se usa en  
calcular el coseno.  
'''  
import time, math
```

```
NCalculos = 100000 # N veces que calcularemos el coseno

# Medimos el tiempo de cpu antes de empezar
tic = time.process_time()

for i in range(NCalculos):
    math.cos(i)

# Medimos el tiempo de cpu al terminar
tac = time.process_time()

# Tiempo medio de cada coseno
dt = (tac - tic)/NCalculos

print('En esta máquina cada coseno tarda %e s de CPU' % dt)
```

2. Ejecute el programa:

```
$ python pruebacos.py
```

3. La respuesta será:

```
En esta máquina cada coseno tarda 1.562500e-07 s de CPU
```

6.8.3. Módulo sys

Este módulo nos proporciona funciones para interactuar con el sistema operativo. Algunas interesantes son:

- `sys.platform`: es una cadena con el nombre del sistema operativo.
- `sys.exit(n)`: acaba el programa, devolviendo un número entero `n` al sistema operativo como código de finalización.
- `sys.argv`: es una lista con los argumentos que hemos utilizado al llamar al intérprete de Python. Sirve para pasar información a nuestro programa. El primer elemento de la lista es siempre el nombre de nuestro programa.

Ejemplo 6.8.3. Ejemplo de uso de `sys.argv`

1. Tecleemos el fichero `param.py`:

```
import sys

for i, a in enumerate(sys.argv):
    print(i, a)
```

2. Ejecute:

```
$ python3 param.py 1.2 'y=2*log(x)' 'mi nombre'
```

3. la respuesta será:

```
0 param.py
1 1.2
2 y=2*log(x)
3 mi nombre
```

6.8.4. Módulo `os`

Este módulo proporciona una forma portátil de utilizar la funcionalidad dependiente del sistema operativo y es una especie de añadido avanzado del módulo `sys` (que contiene algunas órdenes básicas). Algunas funciones interesantes son:

- `os.mkdir(direc)`: crea un directorio con nombre `direc` (que debe contener una tira de caracteres). Si ya existía un fichero con ese nombre nos producirá un error.
- `os.chdir(nombre)`: hace que el directorio de trabajo pase a ser el valor de la variable `nombre` (que debe contener una tira de caracteres). Si no existe ese directorio nos producirá error.
- `os.listdir(direc)`: crea una lista con los nombres de los ficheros del directorio `direc` (una tira de caracteres). Si no se especifica esta variable entonces se usa el directorio actual . (punto).
- `os.system(orden)`: ejecuta el comando expresado en `orden` (una tira de caracteres) en un nuevo terminal. La orden puede crearse con anterioridad o expresarse directamente como una tira de caracteres constante.

Ejemplo 6.8.4. *Ejemplo de uso de `os.system` para cambiar de nombre un fichero existente.*

Ejecutemos python en modo interactivo:

```
$ python3
>>> import os
>>> os.system("mv antiguo nuevo")
>>> os.system("ls -l n*")
>>> exit()
```